CoreFlow: Context-Centric Resource Management for LLM-based Agentic Workflows

***Abstract*—The rise of agentic workflows, which coordinate multiple LLM-based agents to tackle complex tasks, has shifted the LLM serving toward a multi-step, context-driven execution paradigm. This shift introduces highly skewed context-length distributions and agent-specific KV cache-reuse patterns in LLM workloads, challenging existing resource management systems that remain oblivious to these execution traits.**

This paper introduces CoreFlow, a context-centric resource management system designed specifically for the distinct execution patterns of agentic workflows. CoreFlow addresses the limitations of existing works with three key designs: (1) hierarchical performance modeling, which explicitly captures the effects of context-length variance and imbalanced cache reuse inherent in context-driven execution; (2) dual-level resource optimization, combining workflow-level allocation to distribute resources across agents with agent-level partitioning to isolate requests with heterogeneous context lengths; and (3) adaptive cache management, which ensures partition-aligned cache placement while minimizing cache swap overhead. Evaluations on representative workflows demonstrate that CoreFlow reduces end-to-end latency by 57.6% and improves workflow throughput by 63.1% compared to state-of-the-art baselines.

I. Introduction

The rapid advancement of large language models (LLMs) has driven a fundamental shift toward agentic workflows, in which multiple LLM-based agents are coordinated to solve complicated tasks. In this paradigm, each agent is equipped with a specialized LLM, a role specification for domain-specific reasoning, and a set of tools for interacting with external environments. This architecture has rapidly gained traction in both industry and open-source communities, powering diverse applications such as general-purpose AI assistants [1], [2], web-browsing frameworks [3], and coding systems [4], [5].

At the core of agentic workflows lies a **context-driven execution paradigm**, in which each LLM request incorporates extensive execution history to sustain long-term reasoning, causing context to accumulate and cascade across multi-turn interactions. This accumulation manifests in diverging patterns across different agents. Specifically, worker agents, which execute concrete tasks, typically append new context sequentially to preserve temporal continuity. In contrast, orchestrator agents, which decompose and coordinate tasks, often restructure or distill historical outputs from other agents to facilitate global reasoning. These divergent patterns reflect the fundamental distinctions in roles and functionalities among agents.

This context-driven paradigm fundamentally reshapes LLM workloads, introducing two distinctive properties. (1) **Highly skewed context lengths.** As context accumulates over time, early-stage requests typically contain limited historical information, whereas later steps incorporate increasingly rich execution traces. Moreover, varying task complexities lead to differing numbers of reasoning steps, further amplifying disparities in context growth. Consequently, this accumulation leads to highly skewed, heavy-tailed distributions of context lengths. Our analysis on widely adopted workflows [3], [6]–[8] shows that the 99th-percentile context length can exceed the median by more than 60×. (2) **Imbalanced cache reuse.** Successive requests often share substantial common context, offering significant opportunities for cache reuse. Our trace analysis across workflows shows that across more than half of LLM requests, over 60% of the prompt KV cache can be reused from prior requests, significantly reducing prefill computation [9]. However, the potential to exploit cache reuse can vary across agents. Specifically, worker agents that adopt incremental expansion preserve long shared prefixes and achieve a reuse proportion exceeding 90%. In contrast, orchestrator agents that frequently restructure prompts disrupt prefix continuity and exhibit much lower reuse potential. These differences in reuse potential directly translate into disparities in the effective computational load across agents.

These properties expose a critical resource management [1] problem unique to agentic workflows. Unlike traditional LLM serving, where requests are independent, agentic workloads are context-driven and multi-step, exhibiting extreme length variance among requests and imbalanced reuse across agents. This raises a critical question: **How can we orchestrate resources (1) across agents with imbalanced reuse potential and (2) among requests with vastly different context lengths within each agent, to maximize end-to-end workflow performance?**

Unfortunately, existing resource management systems [10]–[14] remain oblivious to the agent-specific workload properties, leading to two fundamental inefficiencies. First, current systems colocate LLM requests on shared resources within each agent to improve utilization. However, the pronounced heterogeneity in context lengths in agentic workloads exacerbates the padding requirements in modern attention implementations [15]–[17], introducing substantial inter-SM load imbalance and synchronization overhead. Our analysis reveals that batching requests with heterogeneous lengths can increase the latency of attention computation by more than 3.3× compared to serving them separately. Second, resource provisioning remains tied to raw prompt length, overlooking the imbalanced cache reuse across agents. This approach disproportionately allocates resources to

[1] In this paper, resource management targets GPU resources in homogeneous clusters and is performed at the granularity of whole GPUs.

agents with extensive context but high reuse potential, resulting in the under-provisioning of low-reuse agents.

To address these limitations, we present CoreFlow, a context-centric resource management system tailored for LLM-based agentic workflows. CoreFlow establishes a hierarchical performance model that leverages inter-request interference quantification and cache-reuse analysis to evaluate agent throughput. Building upon this throughput foundation, CoreFlow executes a dual-level resource optimization: at the agent level, it partitions requests with heterogeneous lengths to dedicated instances to mitigate interference; at the workflow level, it orchestrates resources across agents to maximize the end-to-end throughput. To operationalize these decisions at runtime, CoreFlow implements an adaptive cache management mechanism that dynamically migrates KV caches between instances to align cache placement with resource partitioning decisions while predictively retaining caches in the GPU memory during tool calls to mitigate cache swapping.

We have implemented a prototype of CoreFlow on top of vLLM [18] and conducted extensive experiments across multiple representative agentic workflows and LLMs on both NVIDIA A6000 and A100 clusters. Experimental results show that CoreFlow consistently outperforms state-of-the-art resource management baselines, reducing end-to-end latency by 57.6% and improving throughput by 63.1%. In summary, this paper makes the following contributions:

- We conduct an in-depth characterization of LLM workloads in agentic workflows, revealing highly skewed context lengths and imbalanced cache reuse. Our analysis demonstrates that these properties expose fundamental limitations in existing resource management systems.
- We propose CoreFlow, a resource management system that integrates interference- and reuse-driven performance modeling, dual-level resource optimization, and adaptive cache management to enhance end-to-end performance for agentic workflows.
- We implement CoreFlow atop vLLM and evaluate it across diverse applications and models, demonstrating its effectiveness in improving end-to-end throughput and latency over state-of-the-art systems.

II. Background

A. LLM Inference

Large Language Models (LLMs) are predominantly built on the Transformer architecture [15], utilizing the attention module to model inter-token dependencies. LLM inference follows an autoregressive generation, divided into two distinct phases: (1) the prefill phase, which processes the prompt to generate the initial token, and (2) the decode phase, which iteratively produces subsequent tokens predicated on the entire prefix. To eliminate redundant computation, the intermediate states of processed tokens are preserved in GPU memory as KV caches, allowing subsequent decoding iterations to reuse prior computation. To further enhance efficiency, recent studies [9], [18]–[20] have introduced inter-request cache reuse. In these approaches, KV caches of completed requests are temporarily offloaded to alternative storage media (e.g., CPU memory) and selectively reloaded into GPU memory when shared content—typically matched prefix—is detected. These techniques have become fundamental building blocks of production-scale LLM serving systems [19], [21]–[23].

B. LLM-based Agentic Workflow

Recently, the paradigm for building AI applications has increasingly shifted toward agentic workflows, where multiple LLM-based agents collaborate to complete complex, multi-step tasks. To serve a user query, the workflow initiates a set of **agent invocations**: execution units of individual agents that encapsulate multiple interdependent LLM requests and tool calls without activating other agents. The workflow orchestrates these invocations and synthesizes their outputs to produce the final response to the query.

Fig. 1 illustrates three representative agentic workflows, with constituent agents categorized into two functional roles: **Orchestrator agents** (e.g., Plan and Reflect agents) organize the overall workflow and coordinate results, while **Worker agents** (e.g., Task, Search, Code, and Doc agents) execute specific tasks carry out specific tasks under orchestrator's instruction. The different collaborative mechanisms among these agents create three distinct execution patterns: (1) **ReAct** [7], [24] structures the workflow as an iterative loop interleaving planning and acting. In each iteration, the Plan Agent reviews the status of previous steps to determine the next subtask, while the Task Agent executes it by interacting with external tools. (2) **Reflection** [3] decouples planning and self-reflection into separate agents, enabling more modular functionality. In each iteration, the Plan Agent formulates a search strategy based on feedback from the Reflect Agent and initiates multiple parallel invocations of the Search Agent to retrieve relevant information. The retrieved results are then aggregated and passed to the Reflect Agent for further evaluation and suggestions. (3) **Plan-and-Execute** [6] adopts a more structured paradigm that decomposes the user query into multiple subtasks, organized as a Directed Acyclic Graph (DAG). These subtasks are then dispatched to specialized agents and executed in adherence to the topological constraints defined by the DAG.

III. Motivation

A. Characterizing Workloads of LLM-based Agents

LLM-based agents operate in a context-driven paradigm, incorporating abundant context into each request's prompt to capture complex dependencies among interactions. This context primarily consists of two components: (1) **intra-invocation context**, which encompasses the intermediate outputs generated within the same invocation. For example, the Code Agent alternates between code generation and execution, with each step conditioning on previous code and execution feedback to refine the implementation. (2) **inter-invocation context**, which comprises outputs from the agent's prior invocations. This persistent information ensures consistency across the workflow. For instance, the Plan Agent in the Reflection

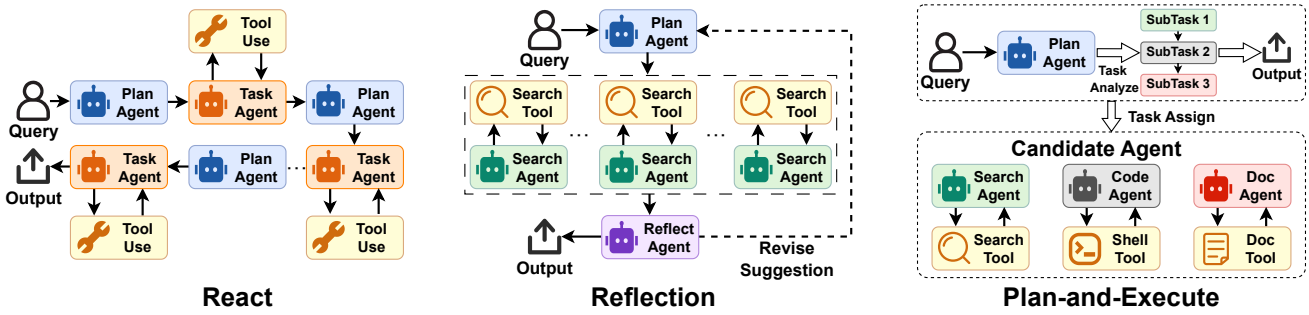
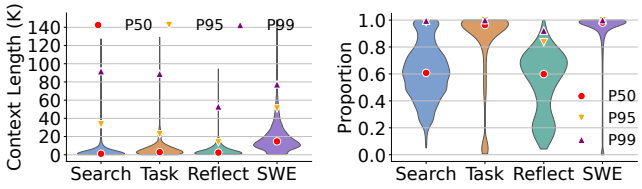
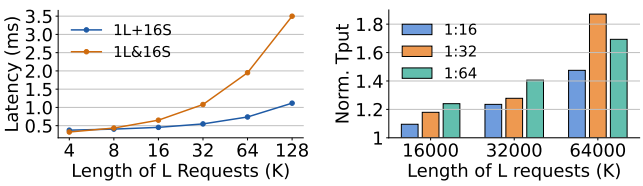


Fig. 1: Representative agentic workflows.



(a) (b)

Fig. 2: (a) Distribution of context lengths across agents. (b) Proportion of cache reuse across requests from different agents.



(a) (b)

Fig. 3: (a) Latency of decode attention. 1L+16S denotes the cumulative latency when serving one long request (L) and 16 short requests (S) separately, while 1L&16S represents the latency when colocating them. (b) Decoding throughput when executing L and S requests separately, normalized to the throughput achieved under colocation. x:x indicates the proportion of L requests relative to S requests.

workflow continuously references previously generated plans to prevent redundant subtask generation. Consequently, the prompt of each request comprises: (1) **context**, which denotes the accumulated trace from prior LLM requests across both intra- and inter-invocation interactions, and (2) **incremental input**, which represents newly introduced content specific to the current step, such as the latest tool-call results.

The context-driven paradigm fundamentally shapes the workloads of agents, giving rise to two prominent properties:

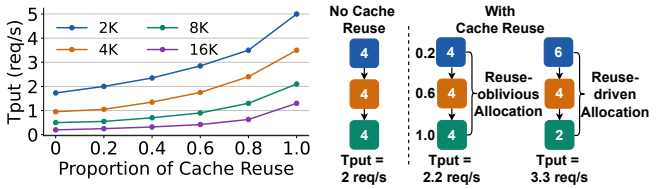
1) Highly Skewed Context Lengths: The dynamic evolution of context during agent execution creates substantial variance in context lengths, leading to a highly skewed distribution. To investigate this, we collect execution traces of different agents by running the workflows described in § II-B using the widely adopted datasets, GAIA [25] and BrowseComp [26]. To capture realistic reasoning behaviors, we employ DeepSeek-V3.2 [27] and Gemini 2.5 Flash [28] for trace collection. Furthermore, we analyze open-source trajectories from SWE-bench [8], a widely adopted benchmark for evaluating LLMs on real-world software issues, to investigate the variance in real-world settings. Figure 2a characterizes the context-length distribution of LLM requests for three representative agents and SWE-bench. The results reveal substantial variability in context lengths, spanning from a few hundred tokens to over 120K tokens. Notably, in the Search Agent, the 99th-percentile length exceeds 90K tokens, surpassing the median (1.5K tokens) by a factor of 60×. These results highlight the pronounced skewness and heavy-tailed nature of agentic workloads.

2) Imbalanced Cache Reuse: Continuous context incorporation introduces substantial shared prefixes across successive LLM requests, providing a prime opportunity to bypass redundant prefill computation by reusing previously computed KV caches. However, the strategies to incorporate context diverge across agents, leading to imbalanced reuse potential. To characterize this divergence, we quantify the cache reusability of each LLM request by measuring the length of the longest common prefix it shares with preceding requests from the same user query. This length is normalized against the total prompt length, yielding a standardized reuse proportion. As illustrated in Fig.2b, agentic workloads manifest substantial reuse potential, with the median proportion exceeding 60% across all evaluated agents. Notably, the Task Agent, which consistently appends new content to its prompt, exhibits a reuse proportion exceeding 90% for most LLM requests. In contrast, the Reflect Agent categorizes and restructures search results to identify information gaps, thereby breaking shared prefixes and reducing the overall reuse potential. Moreover, the Search agent exhibits highly dynamic reasoning depths and prompt lengths, resulting in a broad spectrum of reuse proportions. These findings underscore the substantial and imbalanced nature of cache reuse in agentic workloads.

B. Limitations of Existing Resource Management Systems

Resource management systems have gained widespread adoption [10], [11], [29]–[31] to optimize resource allocation across different agents or inference phases. However, existing approaches overlook the highly skewed context lengths and imbalanced cache reuse inherent in agentic workloads, leading to degraded end-to-end efficiency.

1) Interference-oblivious Request Colocation: Colocating requests with heterogeneous lengths introduces substantial interference, particularly during the decoding phase. This stems from the padding requirement in attention computation, where requests are padded to the maximum sequence length and uniformly partitioned along the length dimension into



(a) (b)

Fig. 4: (a) Request throughput across different prompt lengths and proportions of cache reuse. (b) An example comparing reuse-oblivious and reuse-driven resource allocation. Circles represent agents, fractions denote the reuse proportion, and the integers denote the GPU allocation.

parallelizable blocks to maximize SM utilization. Although padded positions are excluded from attention computation, they incur non-trivial overhead due to inter-SM load imbalance and inter-block synchronization. To quantify the overhead, we evaluate the latency of the FlashAttention kernel [32] under isolated and colocated execution of long (L) and short (S) requests using Llama-3-8B on an NVIDIA A6000 GPU, where S requests are set to 2K tokens (the median observed in the React workflow). As shown in Fig. 3a, colocated execution increases latency by up to 3.3× compared to isolated execution. Similar trends persist in other attention implementations, including FlashInfer, FlexAttention, and Triton-based kernels [17], [33], [34], where latency increases by up to 2.64×, 1.35×, and 5.85×, respectively. These results underscore the substantial performance impact introduced by the interference.

The interference among requests with heterogeneous lengths renders existing resource management systems, which typically colocate these requests, inefficient for serving agentic workloads with highly skewed context lengths. To quantify this inefficiency, we evaluate decoding throughput using the Llama-3-8B model, comparing colocated execution against isolated execution on dedicated instances. In the isolated setting, GPU resources are partitioned proportionally to the memory requirements of each request type. For instance, for a workload with a 1:32 ratio of L (32K tokens) to S (2K tokens) requests, we allocate 1 GPU to execute L requests and 2 GPUs for S requests. As illustrated in Fig. 3b, isolating requests achieves up to an 80% throughput gain, with the advantage expanding as the length of L requests grows. This result demonstrates the substantial performance degradation caused by interference-oblivious request colocation in existing systems.

2) Reuse-oblivious Resource Allocation: KV cache reuse offers a pivotal opportunity to alleviate the prohibitive prefill overhead in long-context execution, particularly in agentic workloads with deep interaction histories and substantial context overlap. To examine the benefits of cache reuse, we evaluate the request throughput of the Llama-3-8B model on an NVIDIA A6000 GPU under varying prompt lengths and reuse proportions. The output length for each request is configured to 106 tokens, reflecting the average value observed in the ReAct workflow. As shown in Fig. 4a, the throughput increases rapidly with the reuse proportion. Particularly at a 16K prompt length, enabling complete cache reuse yields a 6.5× throughput

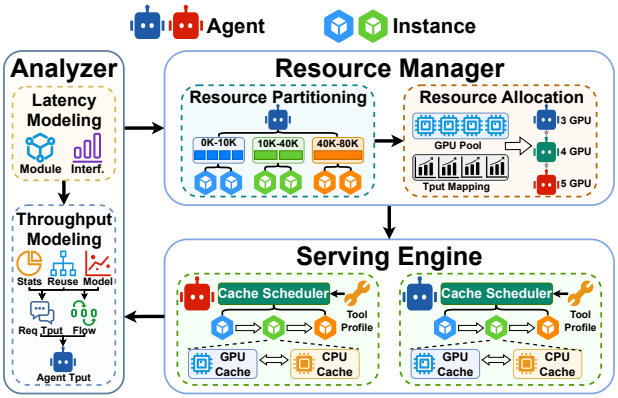


Fig. 5: The architecture overview of CoreFlow.

improvement over a baseline with zero reuse.

Despite the extensive exploration of cache-reuse techniques [19], [20], [35], a critical misalignment persists between these advances and resource management strategies. Specifically, current systems remain reuse-oblivious, rigidly allocating resources based on raw prompt lengths. Given the substantial and imbalanced cache reuse in agentic workloads, these approaches consistently misestimate each agent's computational demands, leading to a resource-demand mismatch: agents with long context but high reuse ratios are over-provisioned despite their marginal prefill costs, while agents with low reuse opportunities are starved. This mismatch inevitably results in severe resource inefficiency and performance degradation.

Fig. 4b illustrates the limitation of reuse-oblivious resource allocation. In this scenario, all agents exhibit identical prompt lengths (8K) and output lengths (106) but differ in their reuse proportions. While a uniform resource distribution is optimal in the absence of cache reuse, it becomes suboptimal once reuse is enabled. In this setting, the agent with the minimal reuse proportion incurs the heaviest prefill load, emerging as the workflow bottleneck that constrains overall throughput. In contrast, the reuse-driven approach strategically rebalances resources toward the bottleneck agent, achieving a 1.5× improvement in end-to-end throughput.

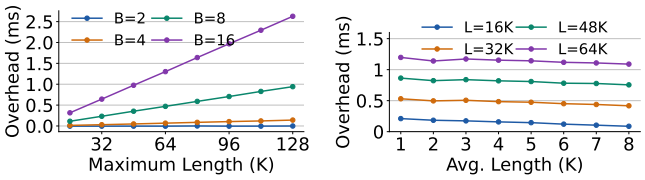
C. Opportunities

The limitations of existing systems highlight the need to account for the unique characteristics of agentic workloads in resource management. At the agent level, resources should be partitioned among requests with heterogeneous lengths to mitigate interference. At the workflow level, resource allocation should be driven by actual compute demands under cache reuse, enabling the precise identification and resolution of workflow bottlenecks. This dual-level optimization ultimately unlocks significant gains in end-to-end performance.

IV. Design Overview

Fig.5 depicts the system architecture of CoreFlow. It operates through the seamless collaboration of three components.

The *Analyzer* conducts performance analysis for each agent, providing a pivotal foundation for resource management. It constructs latency models for LLM modules and interference



(a) (b)

Fig. 6: The overhead of interference on Llama-3-8B. (a) Overhead increases linearly with the maximum request length in the batch. B denotes the batch size. (b) Overhead decreases linearly with the average length of the remaining requests. L denotes the maximum request length.

introduced by heterogeneous request lengths. With these models, it performs hierarchical throughput modeling that first estimates request-level throughput based on request-length statistics and cache-reuse profiles, and subsequently analyzes the request flow to derive the agent throughput.

Building on the performance analysis, the *Resource Manager* performs the dual-level resource optimization. At the agent level, it partitions requests by context lengths and constructs throughput mappings under varying candidate GPU budgets for each agent. At the workflow level, it allocates GPU resources among agents to maximize end-to-end throughput.

The *Serving Engine* serves as the foundation for LLM execution, centering on dynamic KV cache management via an integrated cache scheduler. To reduce cache swapping overhead, the scheduler adaptively retains KV caches in GPU memory during tool execution based on profiled tool-call latency. Meanwhile, it proactively migrates KV caches of context among instances in accordance with the partitioning decisions of the *Resource Manager*.

V. Agent Performance Analysis

A. Latency Modeling

The *Analyzer* first profiles the latency of each iteration in LLM inference. Given the distinct characteristics of LLM modules in the Transformer architecture, it models the per-iteration latency of each module separately. Specifically, the attention module computes correlations between each token and its entire prefix. Accordingly, the *Analyzer* models the latency of this module as a linear function of the number of batched tokens (i.e., the batch size of decode requests or the chunk size of prefill requests) and the aggregated prefix length across tokens [36], [37]. In contrast, other modules process each token independently of preceding tokens. Thus, their latency is modeled as a piecewise-linear function of the number of batched tokens, capturing the tile quantization effect inherent in GPU execution [38]. However, these models fail to capture the decode-phase interference caused by heterogeneous request lengths, leading to significant underestimation.

To quantify the impact of interference, we define **interference overhead** as the additional latency incurred by the attention module when colocating the longest requests with other requests, relative to the baseline latency when executing them in isolation. We identify that, under a fixed decode batch size, the overhead is governed by two factors: (1) the maximum request length within the batch, which amplifies the padding requirement; and (2) the average length of the remaining requests, where a larger average reduces inter-request heterogeneity. As illustrated in Fig.6a, when the average length of the remaining requests is fixed, the overhead scales linearly with the maximum length within the batch. Conversely, as shown in Fig. 6b, the overhead decreases linearly as the average length of the remaining requests increases. Since no consistent correlation is observed between decode batch size and interference overhead, CoreFlow individually profiles the overhead across all potential batch sizes, formulated as:

$$\phi_B(C) = \alpha_B \cdot C_{max} - \beta_B \cdot \left(\frac{(\sum_{i=1}^{B} C_i) - C_{max}}{B-1}\right) + \gamma_B \quad (1)$$

Here, B denotes the batch size, C denotes the length distribution of requests in the batch, and C_{max} is the maximum length. α_B, β_B, and γ_B denote the related coefficients corresponding to the batch size B. The *Analyzer* employs offline profiling to calibrate the coefficients of latency and interference models for each serving phase and agent. These models are then integrated to predict per-iteration inference latency, which in turn serves as the basis for evaluating agent throughput.

B. Hierarchical Throughput Modeling

For a specific agent a, given its workload statistics analyzed from the historical traces and a partition $R = [e_l, e_u)$ that specifies a request-length range bounded by minimum e_l and maximum e_u, the *Analyzer* evaluates the agent throughput of a single serving instance. In contrast to traditional LLM-based applications evaluated via token-level or request-level metrics, agents operate at the granularity of invocations, which typically comprise multiple successive LLM requests. To capture this pattern, the *Analyzer* quantifies the **Sustainable Invocation Throughput** (λ_R^a), defined as the upper bound of invocation rate subject to the constraint that the resulting rate of LLM requests in R does not surpass an instance's serving capacity. Owing to the intricate dependencies between LLM requests and agent invocations, the *Analyzer* adopts a hierarchical modeling methodology. Specifically, it first establishes a lightweight evaluation of the per-instance throughput of LLM requests T_R^a. Subsequently, it captures the request flow across partitions to formally derive λ_R^a from the request-level capacity.

1) Request-level Throughput Evaluation: To assess T_R^a, the *Analyzer* first evaluates the cache reusability for each LLM request to derive the effective prefill load. Specifically, for each request, the *Analyzer* computes the longest common prefix with its preceding requests within the same invocation to identify the reusable prompt. To further capture inter-invocation reuse, the *Analyzer* evaluates the prefix overlap between the first request of an invocation and the final request of its immediate predecessor (the temporally adjacent invocation). Notably, due to the stochastic arrival of user queries and the limited prefix overlap among their requests [9], the *Analyzer* excludes cross-query cache reuse from the model. Consequently, each LLM request is characterized by three key metrics: (1) incremental

length, defined as the number of new tokens requiring prefill computation; (2) output length, denoting the number of tokens to be generated; and (3) reuse length, representing the number of tokens whose KV caches can be reused from prior requests.

Using these cache-reuse profiles, the *Analyzer* then evaluates the request-level throughput T_R^a. Due to the stochastic nature of request arrivals and the inherent variance in request lengths, accurate quantification of T_R^a remains non-trivial. While prior efforts adopt event-driven simulators [11], [39] for throughput estimations, these methods incur non-negligible runtime overhead, limiting their feasibility for rapidly exploring the design space. To achieve efficient throughput evaluation, the *Analyzer* uses a lightweight approximation model. Specifically, the *Analyzer* calculates the average incremental, output, and reuse lengths of LLM requests within R. To evaluate a candidate throughput selected via binary search, it determines the maximum batch size based on these statistics and GPU memory constraints, while ensuring that the per-second prefill load can be completed under chunked prefill execution. The per-second inference is then decomposed into three distinct phases: (1) loading reused KV caches, (2) chunked prefill for uncached tokens, and (3) autoregressive decoding. By assessing the duration of these decoupled phases, the *Analyzer* derives the generation rate of decode tokens. If this rate satisfies the per-second decoding demand (i.e., the product of the target throughput and the average output length), the candidate throughput is deemed feasible, allowing the search to explore higher values. Conversely, the search contracts toward lower targets. This iterative process refines the candidate until it converges to the maximum request capacity.

2) Invocation-level Throughput Evaluation: Leveraging T_R^a, the *Analyzer* analyzes the request flow across different partitions to derive λ_R^a. Specifically, the aggregate request-level rate within R comprises two components: (1) **Entry Load**, consisting of requests newly entering R (i.e., initial requests of invocations or requests whose all predecessors fall outside R). Since an invocation contributes at most one entry request in R, the entry load is equal to the rate of invocations that access R. Given P_R^a as the proportion of invocations that contain at least one request within R, the entry load is calculated as $\lambda_R^a \cdot P_R^a$. (2) **Recurrent Load**, corresponding to requests with predecessors falling within R. Given Q_R^a as the proportion of such requests in R, this component is defined as $T_R^a \cdot Q_R^a$. To maintain system stability, the request-level rate must be bounded by the instance capacity, formulated as:

$$T_R^a \geq T_R^a \cdot Q_R^a + \lambda_R^a \cdot P_R^a \tag{2}$$

By rearranging this formulation, the *Analyzer* derives the sustainable invocation throughput as $\lambda_R^a = \frac{T_R^a(1-Q_R^a)}{P_R^a}$. This closed-form formulation efficiently translates request-level capacity into invocation-level throughput.

VI. Workflow Resource Management

The *Resource Manager* maximizes workflow throughput via a dual-level optimization: (1) **Workflow-level resource allocation**, which determines the GPU allocation among agents; and (2) **Agent-level resource partitioning**, which divides requests of heterogeneous lengths and assigns GPUs to each partition within each agent. Since the incremental input and output of LLM requests are challenging to forecast in advance, partitioning is performed based on context lengths, which become available upon the completion of preceding requests.

TABLE I: Key notations

Notation	Definition
N	Total GPU budget in the cluster
A	Set of agents in the workflow
E	Maximum context lengths of agents
G	Maximum number of partitions
Δ_a	Number of GPUs per instance of agent a
M_a	Resource-throughput mapping of agent a
I_a	Number of invocations per user query of agent a
T_R^a	Per-instance request-level throughput of agent a
λ_R^a	Sustainable invocation throughput per instance of agent a given partition R
δ	Step size of context lengths

This joint optimization problem exhibits an intractable circular dependency. Specifically, determining the optimal allocation depends on the per-agent throughput profile, which is governed by the partitioning scheme. Conversely, the optimal partitioning scheme is constrained by the allocated GPUs. To decouple this dependency, the *Resource Manager* first profiles the resource-throughput mapping by pre-computing optimal partitioning schemes across all potential GPU budgets for each agent. It then optimizes the GPU allocation among agents using these established mappings as a pivotal reference.

A. Agent-level Resource Partitioning

For a specific agent a, the resource partitioning can be formulated as a dynamic programming (DP) problem over GPU budgets, context lengths, and partition counts. However, given the total GPU budget N, the maximum context length E, and the maximum partition count G (configured to 3 by default), the computational complexity of this approach scales as $O(N^2E^2G)$, which becomes intractable for large-scale clusters or workloads with heavy-tailed context lengths. To ensure scalability, the *Resource Manager* decouples the process into two sequential stages, as detailed in Algorithm 1.

In the first stage, the *Resource Manager* optimizes the context partitioning scheme by segmenting the entire context-length space into multiple partitions, each served by a dedicated GPU set. To achieve this, the *Resource Manager* introduces the **Normalized Throughput Cost (NTC)**, defined as the number of GPUs required per unit of throughput. Specifically, given a partitioning scheme s and a total GPU budget n, agent a achieves its maximum throughput θ_a when the sustainable invocation throughput across all partitions is balanced. In this setting, the GPU allocation for partition $R \in s$ is $\frac{\theta_a \cdot \Delta_a}{\lambda_R^a}$, where Δ_a represents the optimal instance size that maximizes request-level throughput based on average workload statistics of agent a. Incorporating the aggregate GPU budget constraint, the NTC is derived as $\frac{n}{\theta_a} = \sum_{R \in s} \frac{\Delta_a}{\lambda_R^a}$.

Leveraging the defined metric, the *Resource Manager* adopts a lightweight DP to optimize the context partitioning scheme.

Let $dp[e][g]$ denote the minimum NTC when partitioning requests with a maximum context length e into g partitions. The *Resource Manager* populates the DP table by enumerating all partition counts and context-length boundaries, driven by the following transition function:

$$dp[e][g] = \min_{e^* \in \{0,\ldots,e-\delta\}} \left\{ dp[e^*][g-1] + \frac{\Delta_a}{\lambda^a_{[e^*,e)}} \right\} \quad (3)$$

To maintain computational efficiency, context-length boundaries are evaluated with a step size δ (configured to 2000 by default). Simultaneously, $parent[e][g]$ is maintained to record the preceding boundary e^* that yields the minimum cost for each boundary e and partition count g. Finally, the *Resource Manager* backtracks through the $parent$ table to identify optimal context partitioning schemes S.

In the second stage, the *Resource Manager* constructs the resource-throughput mapping M_a. Specifically, for each GPU budget n and each partition count g, the *Resource Manager* first initializes the GPU allocation $\Lambda[R]$ for each partition $R \in S[g]$ proportional to the λ^a_R. To strictly adhere to the budget n, it employs a greedy adjustment strategy, iteratively reclaiming instances from the partition R^* that achieves the highest throughput after the adjustment. Once the allocation is finalized, the agent throughput under partition count g is determined by the bottleneck partition, and $M_a[n]$ is updated by selecting the optimal partition count. Notably, this two-stage algorithm reduces the computational complexity from $O(N^2E^2G)$ to $O(N^2G + E^2G)$, significantly alleviating the overhead of resource partitioning.

B. Workflow-level Resource Allocation

Utilizing the resource-throughput mappings, the *Resource Manager* optimizes the GPU allocation across agents to maximize end-to-end throughput. Since the workflow throughput is fundamentally dictated by the bottleneck agent, which exhibits the minimum throughput under a given allocation, the *Resource Manager* adopts an iterative strategy that progressively relieves the current bottleneck. Algorithm 2 outlines the resource allocation procedure. Initially, each agent is initialized with one serving instance to ensure minimal serving capacity. In each iteration, agents are ranked in ascending order by their throughput normalized by the number of invocations. The agent with the minimum throughput that can fit within the remaining GPU budget is then selected and allocated one additional serving instance. The procedure terminates once the GPU budget is exhausted, yielding the final allocation scheme.

Algorithm 1: Agent-level Resource Partitioning

Function `Partitioning(`N, E, G, λ^a, Δ_a, δ`)`**:**
- `// First stage`
- $dp[0, \delta, \ldots E][0, 1, \ldots G] \leftarrow \infty$;
- $dp[0][0] \leftarrow 0$;
- $parent[0, \delta, \ldots E][0, 1, \ldots G] \leftarrow 0$;
- **for** $g \in \{1, 2, \ldots, G\}$ **do**
  - **for** $e \in \{\delta, 2\delta, \ldots, E\}$ **do**
    - **for** $e^* \in \{0, \delta, \ldots, e - \delta\}$ **do**
      - $cost \leftarrow dp[e^*][g-1] + \frac{\Delta_a}{\lambda^a_{[e^*,e)}}$;
      - **if** $cost < dp[e][g]$ **then**
        - $dp[e][g] \leftarrow cost$;
        - $parent[e][g] \leftarrow e^*$;
- **for** $g \in \{1, 2, \ldots, G\}$ **do**
  - $S[g] \leftarrow$ Backtrack$(parent, E, g)$;
- `// Second stage`
- **for** $n \in \{\Delta_a, 2\Delta_a \ldots, N\}$ **do**
  - $M_a[n] \leftarrow 0$;
  - **for** $g \in \{1, 2, \ldots, \min(G, \frac{n}{\Delta_a})\}$ **do**
    - $\Lambda[R] \leftarrow \lceil \frac{\lambda^a_R \cdot n}{\sum_{r \in S[g]} \lambda^a_r} \rceil, \quad \forall R \in S[g]$;
    - **while** $\sum_{R \in S[g]} \Lambda[R] > n$ **do**
      - $R^* \leftarrow \text{argmax}_{R \in S[g]} \lambda^a_R \cdot \frac{\Lambda[R] - \Delta_a}{\Delta_a}$;
      - $\Lambda[R^*] \leftarrow \Lambda[R^*] - \Delta_a$;
    - $\Lambda_{min} \leftarrow \min_{R \in S[g]} \lambda^a_R \cdot \frac{\Lambda[R]}{\Delta_a}$;
    - $M_a[n] \leftarrow \max(\Lambda_{min}, M_a[n])$;
- **return** M_a;

Algorithm 2: Workflow-Level Resource Allocation

Function `Allocation(`A, M, I, Δ, N`)`**:**
- $n_a \leftarrow \Delta_a, \quad \forall a \in A$;
- **while** $\sum_{a \in A} n_a < N$ **do**
  - Sort A in ascending order by $\frac{M_a[n_a]}{I_a}$;
  - **for** $a^* \in A$ **do**
    - **if** $N - \sum_{a \in A} n_a \geq \Delta_{a^*}$ **then**
      - $n_{a^*} \leftarrow n_{a^*} + \Delta_{a^*}$;
      - **break**;
- **return** $\{n_a | a \in A\}$;

VII. Adaptive Cache Management

Intra-instance Cache Swap. To mitigate frequent cache swapping, CoreFlow employs a predictive keep-alive mechanism. Specifically, for completed requests that trigger tool calls, CoreFlow pins the KV cache in GPU memory when the eviction overhead exceeds the potential gain—defined as the difference between the expected tool-call latency and the swap time of the next request. The keep-alive duration is set to match the tool-call latency. If no tool call is identified or the duration expires, the cache can be evicted to CPU memory under memory pressure to release GPU capacity while preserving opportunities for subsequent reuse.

Inter-instance Cache Migration. CoreFlow continuously tracks cache availability within each instance. Upon receiving an LLM request, it first searches for a GPU-resident cache hit across instances of the corresponding agent. If a GPU hit is detected, the request is routed directly to the corresponding instance to minimize retrieval latency. Otherwise, CoreFlow

TABLE II: Workload statistics of each agent. PAE refers to the Plan-and-Execute workflow.

Workflow	Agent	Prompt	Output	Requests	Invocations
React	Plan	166.9	70.7	1	6.4
React	Task	499.1	113.5	2.8	6.4
Reflection	Plan	1053.6	145.9	1	5.9
Reflection	Search	110.8	782.1	1	12.6
Reflection	Reflect	3036.2	287.5	1	5.9
PAE	Plan	880.7	402.2	3.2	1
PAE	Search	1165.2	178.6	8.3	2.4
PAE	Code	984.3	573.5	7.6	1.4
PAE	Doc	3104.5	303.8	3.7	1.1

dispatches the request to the instance with the longest matching prefix in CPU memory. To align scheduling with the partitioning scheme determined by the *Resource Manager*, CoreFlow proactively migrates the KV cache of a completed request to the appropriate partition once its length exceeds the current boundary. This mechanism ensures that subsequent requests are seamlessly routed to the proper instance.

VIII. Implementation

CoreFlow is a cluster-wide resource management system tailored for agentic workflows, implemented with approximately 5,000 lines of Python code.

Frontend and Monitoring. CoreFlow exposes a frontend interface allowing developers to define model configurations, role descriptions, and external toolsets for each agent. To support request submission, it extends OpenAI's Chat Completion interface by tagging each request with a designated agent ID, a $query_id$ for end-to-end tracking, and a $subtask_id$ to group dependent LLM requests within a specific invocation. Additionally, CoreFlow implements a lightweight monitoring module to collect runtime metrics, including arrival and completion timestamps, prompts and outputs, and tool-call profiles for each request. These statistics are persisted in Redis [40], a centralized repository accessible by all CoreFlow's components for real-time orchestration.

Serving Engine. CoreFlow utilizes vLLM (v0.9.2) as the underlying engine for LLM serving, adopting the FlashAttention kernel for attention computation. The engine employs a query-level First-Come-First-Served (FCFS) scheduling policy, prioritizing earlier-arriving user queries [41], [42]. Consistently, under memory pressure, KV caches are evicted in the same query-level FCFS order, while caches within the same query are managed using a Least Recently Used (LRU) policy.

IX. Evaluation

A. Experimental Setup

Models and Hardware: We evaluate CoreFlow using two representative models: Llama-3-8B and Qwen2.5-32B. Llama-3-8B is deployed on a cluster with four physical hosts, each equipped with eight NVIDIA RTX A6000 GPUs. Qwen2.5-32B is evaluated on a larger cluster with the same number of hosts, each provisioned with eight NVIDIA A100 GPUs interconnected via NVLink. To facilitate cache reuse, each GPU is provisioned with 120GB of CPU memory.

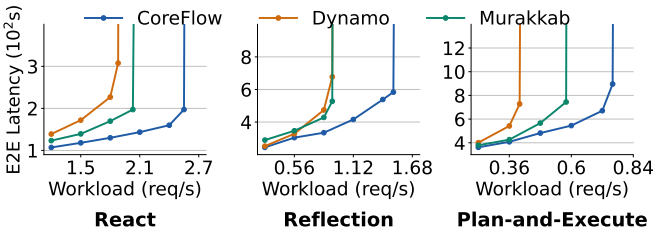


Fig. 7: Average end-to-end latency using Llama-3-8B.

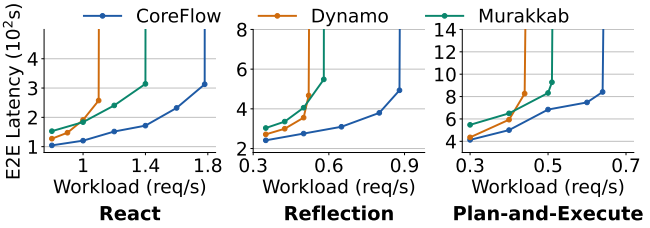


Fig. 8: Average end-to-end latency using Qwen2.5-32B.

Workloads: We evaluate CoreFlow using three agentic workflows as detailed in § II-B. Table II summarizes the average workload statistics of each agent, including the prompt and output lengths, the number of LLM requests per invocation, and the number of invocations per user query. To reproduce these workloads, we align each request's prompt with the collected traces and ensure that the generated tokens match the recorded outputs. Workloads are generated using a Poisson distribution with different arrival rates.

Baselines. We compare CoreFlow against two state-of-the-art resource management systems.

- **Murakkab [10]**: This scheme formulates the resource allocation for the agentic workflow as a mixed integer linear programming (MILP) problem to maximize end-to-end throughput. For each agent, LLM requests are colocated in shared instances.
- **DynamoLLM [12]**: This approach partitions LLM requests based on context-length quantiles for each agent, and then distributes resources across agents to optimize the end-to-end throughput.

B. End-to-end Performance

We first evaluate the performance of CoreFlow against baseline schemes, measuring the average end-to-end latency across different models, applications, and workload intensities. As shown in Fig. 7 and Fig. 8, CoreFlow consistently outperforms all baselines, reducing the average end-to-end latency by over 46.3% and 57.6% compared to Murakkab and DynamoLLM, respectively. These performance gains translate into a substantial 63.1% increase in workflow throughput. The performance degradation in Murakkab primarily stems from its interference-oblivious design. Due to the colocation of LLM requests with heterogeneous lengths, the system incurs substantial interference. While DynamoLLM mitigates interference by aggregating requests with comparable lengths, its reuse-oblivious allocation introduces two critical inefficiencies that can even offset the benefits of interference mitigation.

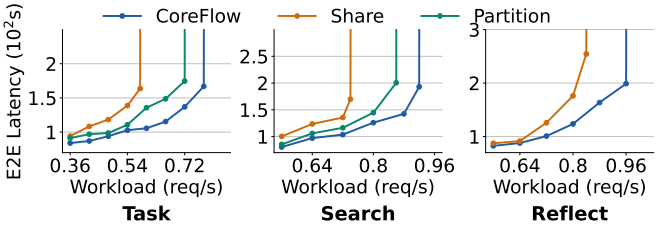


Fig. 9: Benefits of CoreFlow's agent-level designs.

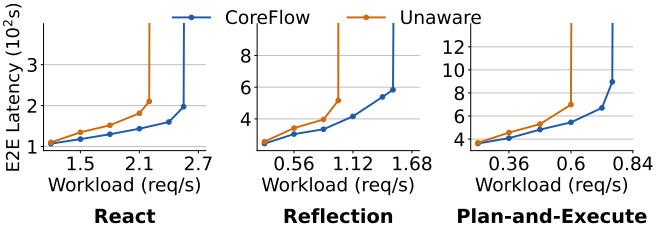


Fig. 10: Benefits of reuse-driven resource allocation.

For each agent, it skews resource allocation toward long-context requests, inevitably underprovisioning short requests. At the workflow level, overlooking cache reuse misdirects resources to non-critical agents, thereby degrading overall performance. In contrast, CoreFlow integrates interference- and reuse-driven performance modeling with dual-level resource optimization, effectively mitigating these inefficiencies and delivering substantial performance gains.

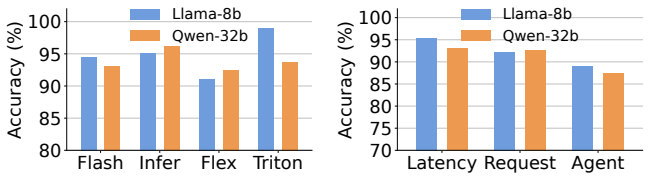
C. Ablation Studies

In this part, we provide an in-depth study of the individual contribution of each component using the Llama-3-8B model.

1) Resource Partitioning and Keep-alive Mechanism: For each agent, CoreFlow leverages resource partitioning to mitigate interference among LLM requests and employs a keep-alive mechanism during tool execution to minimize cache swapping. To validate these designs, we conduct experiments on a single host equipped with eight A6000 GPUs using three representative agents: the Task agent in the React workflow, the Search agent in the Plan-and-Execute workflow, and the Reflect agent from the Reflection workflow. For comparison, we implement two additional baseline schemes: (1) Share, which executes without resource partitioning and the keep-alive mechanism; and (2) Partition, which adopts the resource partitioning. Note that the Partition scheme is excluded for the Reflect agent, as it involves no external tool calls.

As shown in Fig. 9, Partition mitigates interference among requests compared to Share, reducing end-to-end latency by 41.1% and improving throughput by 24.1%. The most pronounced gains are observed in the Task agent, which exhibits the highest skewness in context lengths. Moreover, CoreFlow further improves throughput by 8.3% by preserving KV caches in GPU memory during tool execution to avoid frequent cache swaps. These results demonstrate the effectiveness of CoreFlow's designs for individual agents.

2) Reuse-driven Resource Allocation: CoreFlow derives the actual prefill load by quantifying cache reuse among LLM requests for each agent, enabling more effective workflow-level resource allocation. To demonstrate the effectiveness, we implement an additional baseline that allocates resources based on raw workload statistics while incorporating CoreFlow's agent-level designs. As shown in Fig. 10, CoreFlow improves throughput by 15.9% in the simple ReAct workflow. For the more complex Reflection and Plan-and-Execute workflows, the imbalance in cache reuse across agents becomes more pronounced. In these settings, CoreFlow improves end-to-end throughput by 53.1% and 26.7%, respectively. These results highlight the importance of bridging cache reuse with efficient resource allocation.



(a) (b)

Fig. 11: (a) Prediction accuracy of the interference overhead across attention implementations. (b) Prediction accuracy of the inference latency, request-level throughput, and agent-level throughput.

D. Accuracy Evaluation

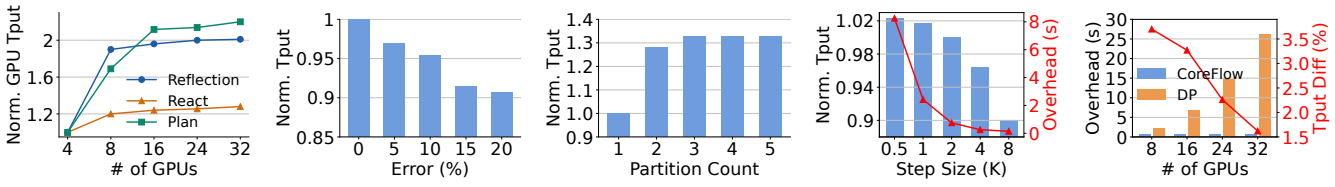
Interference overhead: We first evaluate the accuracy of the interference model, which predicts interference overhead under varying batch sizes and request lengths. As illustrated in Fig. 11a, the model consistently achieves high accuracy, with accuracy ranging from 93% to 99%. Moreover, the model demonstrates strong robustness, maintaining stable performance across diverse attention implementations and models. These results demonstrate CoreFlow's effectiveness in capturing interference among requests with heterogeneous lengths.

Composite performance: Building on the interference model, we further evaluate the prediction accuracy of composite metrics, including the per-iteration latency, request-level throughput, and agent-level throughput. Here, throughput is measured as the average number of completed requests or invocations per minute under a saturated workload. Fig. 11b shows that the prediction error remains tightly bounded as the metrics transition from inference latency to aggregate throughput, staying below 12.6% consistently. This precise modeling provides an important foundation for effective resource management.

E. Sensitivity Analysis

In this part, we comprehensively investigate the robustness of CoreFlow under different system configurations using the Llama-3-8B model. Unless otherwise specified, we report the normalized throughput averaged over all workflows.

1) Cluster Sizes: CoreFlow's effectiveness is closely tied to the total GPU budget available in the cluster. A large cluster allows CoreFlow to precisely partition resources within each agent and balance demand across agents. As illustrated in Fig. 12a, per-GPU throughput, defined as the end-to-end



(a) (b) (c) (d) (e)

Fig. 12: (a)-(d) Normalized throughput across varying system configurations, with the baseline set to 1: (a) cluster size, (b) prediction error rates, (c) maximum partition count, and (d) step size of context lengths. (e) Computational overhead and throughput comparison between dynamic programming (DP) and CoreFlow's algorithm.

throughput normalized by the total GPU budget, increases with the cluster scale across all workflows. Moreover, more complex workflows achieve greater gains from larger clusters. Specifically, the Plan-and-Execute workflow approaches the peak per-GPU throughput at 16 GPUs, whereas others converge at around 8 GPUs. These results demonstrate the importance of effective resource management in large clusters.

2) Accuracy Errors: CoreFlow relies on accurate performance modeling to achieve dual-level optimization. To assess the impact of prediction errors, we manually inject errors into CoreFlow's models and evaluate the resulting allocation strategies. As illustrated in Fig.12b, even with a 20% prediction error, throughput drops by only 9.3%, demonstrating CoreFlow's robustness against prediction inaccuracies.

3) Parameter Configurations: CoreFlow employs two hyperparameters, including the maximum partition count G and the step size of context lengths δ. Increasing G or decreasing δ enables finer-grained resource partitioning to mitigate interference, but also incurs higher search overhead. As illustrated in Fig. 12c, increasing G to 3 improves throughput significantly, but further expansion to 5 yields minor additional gain as the benefits of interference mitigation by increasing partition counts are offset by resource fragmentation. Similarly, as illustrated in Fig. 12d, reducing δ to 2,000 provides a 10% throughput gain, whereas a further reduction to 500 offers only a marginal 2.5% improvement at the cost of 11$\times$ higher computational overhead. Accordingly, CoreFlow sets G to 3 and δ to 2,000 by default.

F. Overhead and Scalability

Compared to employing a dynamic programming (DP) algorithm that exhaustively searches for optimal resource partitioning schemes, CoreFlow adopts a two-stage strategy to reduce computational overhead. As shown in Fig. 12e, CoreFlow incurs low overhead across cluster sizes, while the DP approach reaches 26.05s when scaling to 32 GPUs. This reduction in complexity incurs a throughput penalty of less than 3.7%, with the gap further diminishing in larger clusters.

To ensure scalability in large-scale deployments, CoreFlow parallelizes agent-level resource partitioning via multi-core parallelism, then aggregates per-agent mappings to optimize workflow-level resource allocation. In a simulated environment comprising 2,000 GPUs and 100 agents, this design completes the entire resource management procedure within 5 seconds on a physical host equipped with 16 CPUs. This low-latency execution demonstrates CoreFlow's capability to handle large-scale, complex workflows.

X. Related Work

Cache Reuse: Extensive research has focused on KV cache-reuse techniques for multi-turn LLM interactions [19]–[21], [35], [41], [43]–[45]. Mooncake [19] implements multi-level storage and cross-instance cache migration to mitigate redundant prefill computation. CacheBlend [20] introduces a selective recomputation strategy to integrate precomputed KV caches for retrieval-augmented generation (RAG). ContextPilot [35] introduces a context index to identify context overlap across LLM requests and proposes context ordering and deduplication to maximize cache reuse. CoreFlow is designed to bridge the gap between resource management and cache-reuse mechanisms.

Resource Management: Efficient resource management is pivotal for optimizing serving performance for LLMs [10]–[12], [23], [30], [46]–[51]. DistServe [11] proposes phase-aware resource disaggregation, optimizing resource allocation for distinct phases of LLM inference based on workload statistics and network topologies. DynamoLLM [12] addresses workload heterogeneity by dynamically partitioning requests and tailoring resource configurations for each partition. Murakkab [10] introduces a declarative abstraction to decouple workflow specification from execution, employing an MILP-based optimizer to determine per-agent configurations. Existing solutions remain oblivious to the context-driven execution paradigm inherent in agentic workflows, leading to a mismatch between resource allocation and the actual demands.

XI. Conclusion

The context-driven execution paradigm of LLM-based agentic workflows introduces highly skewed context lengths and imbalanced cache reuse in agentic workloads, rendering conventional resource management systems inefficient. To bridge this gap, we present CoreFlow, a context-centric resource management system built on three core designs: (1) hierarchical performance modeling that explicitly captures interference and cache reuse among requests; (2) resource optimization that combines agent-level resource partitioning and workflow-level resource allocation; and (3) adaptive cache management that aligns cache placement with resource management decisions. Extensive evaluations demonstrate that CoreFlow delivers substantial improvements in both latency and throughput compared to state-of-the-art systems.

REFERENCES

[1] "Openclaw," https://openclaw.ai/, 2026.

[2] G. Wang, S. Cheng, X. Zhan, X. Li, S. Song, and Y. Liu, "Openchat: Advancing open-source language models with mixed-quality data," *CoRR*, vol. abs/2309.11235, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2309.11235

[3] "Gemini fullstack langgraph," https://github.com/google-gemini/gemini-fullstack-langgraph-quickstart, 2025.

[4] "Claude code," https://code.claude.com/docs/en/overview, 2026.

[5] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," *CoRR*, vol. abs/2308.00352, 2023. [Online]. Available: https://doi.org/10.48550/arxiv.2308.00352

[6] "Eigent: The open source cowork desktop to unlock your exceptional productivity," https://github.com/eigent-ai/eigent/tree/main, 2025.

[7] M. Hu, Y. Zhou, W. Fan, Y. Nie, B. Xia, T. Sun, Z. Ye, Z. Jin, Y. Li, Q. Chen *et al.*, "Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation," *CoRR*, vol. abs/2505.23885, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2505.23885

[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" *CoRR*, vol. abs/2310.06770, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2310.06770

[9] J. Wang, J. Han, X. Wei, S. Shen, D. Zhang, C. Fang, R. Chen, W. Yu, and H. Chen, "{KVCache} cache in the wild: Characterizing and optimizing {KVCache} cache at a large cloud provider," *CoRR*, vol. abs/2506.02634, 2025. [Online]. Available: https://doi.org/10.48550/arxiv.2506.02634

[10] G. I. Chaudhry, E. Choukse, H. Qiu, Í. Goiri, R. Fonseca, A. Belay, and R. Bianchini, "Murakkab: Resource-efficient agentic workflow orchestration in cloud platforms," *CoRR*, vol. abs/2508.18298, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2508.18298

[11] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," *CoRR*, vol. abs/2401.09670, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2401.09670

[12] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, "Dynamollm: Designing llm inference clusters for performance and energy efficiency," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1348–1362. [Online]. Available: https://doi.org/10.1109/HPCA61900.2025.00102

[13] Y. Jiang, R. Yan, X. Yao, Y. Zhou, B. Chen, and B. Yuan, "Hexgen: Generative inference of large language model over heterogeneous environment," *CoRR*, vol. abs/2311.11514, 2023. [Online]. Available: https://doi.org/10.48550/arxiv.2311.11514

[14] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132. [Online]. Available: https://doi.org/10.1109/ISCA59077.2024.00019

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: http://arxiv.org/abs/1706.03762

[16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," *CoRR*, vol. abs/2205.14135, 2022. [Online]. Available: https://doi.org/10.48550/arxiv.2205.14135

[17] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy *et al.*, "Flashinfer: Efficient and customizable attention engine for llm inference serving," *CoRR*, vol. abs/2501.01005, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2501.01005

[18] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626. [Online]. Available: https://doi.org/10.1145/3600006.3613165

[19] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: A kvcache-centric disaggregated architecture for llm serving," *CoRR*, vol. abs/2407.00079, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2407.00079

[20] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "Cacheblend: Fast large language model serving for rag with cached knowledge fusion," in *Proceedings of the Twentieth European Conference on Computer Systems*, 2025, pp. 94–109. [Online]. Available: https://doi.org/10.1145/3689031.3696098

[21] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du *et al.*, "Lmcache: An efficient kv cache layer for enterprise-scale llm inference," *CoRR*, vol. abs/2510.09665, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.09665

[22] "Nvidia dynamo platform," https://www.nvidia.com/en-us/ai/dynamo/, 2025.

[23] Y. Xiang, X. Li, K. Qian, Y. Yang, D. Zhu, W. Yu, E. Zhai, X. Liu, X. Jin, and J. Zhou, "Aegaeon: Effective gpu pooling for concurrent llm serving on the market," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025, pp. 1030–1045. [Online]. Available: https://doi.org/10.1145/3731569.3764815

[24] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "Camel: Communicative agents for" mind" exploration of large language model society," *CoRR*, vol. abs/2303.17760, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2303.17760

[25] G. Mialon, C. Fourrier, T. Wolf, Y. LeCun, and T. Scialom, "Gaia: a benchmark for general ai assistants," *CoRR*, vol. abs/2311.12983, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2311.12983

[26] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese, "Browsecomp: A simple yet challenging benchmark for browsing agents," *CoRR*, vol. abs/2504.12516, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2504.12516

[27] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong *et al.*, "Deepseek-v3. 2: Pushing the frontier of open large language models," *CoRR*, vol. abs/2512.02556, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2512.02556

[28] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *CoRR*, vol. abs/2507.06261, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2507.06261

[29] Y. Liang, Z. Chen, P. Zuo, Z. Zhou, X. Chen, and Z. Yu, "Injecting adrenaline into llm serving: Boosting resource utilization and throughput via attention disaggregation," *CoRR*, vol. abs/2503.20552, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2503.20552

[30] N. Pagonas, Y. Chung, K. Kaffes, and A. Krishnamurthy, "Cortex: Workflow-aware resource pooling and scheduling for agentic serving," *CoRR*, vol. abs/2510.14126, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.14126

[31] I. Gim, Z. Ma, S.-s. Lee, and L. Zhong, "Pie: A programmable serving system for emerging llm applications," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025, pp. 415–430. [Online]. Available: https://doi.org/10.1145/3731569.3764814

[32] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," *CoRR*, vol. abs/2407.08608, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2407.08608

[33] J. Dong, B. Feng, D. Guessous, Y. Liang, and H. He, "Flex attention: A programming model for generating optimized attention kernels," *CoRR*, vol. abs/2412.05496, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2412.05496

[34] "vllm triton-based attention kernel," https://github.com/vllm-project/vllm/blob/main/vllm/v1/attention/backends/triton_attn.py, 2025.

[35] Y. Jiang, Y. Huang, L. Cheng, C. Deng, X. Sun, and L. Mai, "Contextpilot: Fast long-context inference via context reuse," 2026. [Online]. Available: https://arxiv.org/abs/2511.03475

[36] J. Liao, Q. Dong, Y. Liang, Z. Zhou, and X. Chen, "Laser: Unlocking layer-level scheduling for efficient multi-slo llm serving," in *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2026, pp. 509–521. [Online]. Available: https://doi.org/10.1145/3774934.3786413

[37] Z. Mo, J. Liao, H. Xu, Z. Zhou, and C. Xu, "Hetis: Serving llms in heterogeneous gpu clusters with fine-grained and dynamic parallelism," *CoRR*, vol. abs/2509.08309, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2509.08309

[38] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," *CoRR*, vol. abs/2403.02310, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2403.02310

[39] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov, "Vidur: A large-scale simulation framework for llm inference," *CoRR*, vol. abs/2405.05465, 2024. [Online]. Available: https://doi.org/10.48550/arxiv.2405.05465

[40] "Redis," https://redis.io/, 2025.

[41] H. Li, Q. Mang, R. He, Q. Zhang, H. Mao, X. Chen, H. Zhou, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live," *CoRR*, vol. abs/2511.02230, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2511.02230

[42] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang, C. Wang, Y. Huang, Z. Chen, J. E. Gonzalez *et al.*, "Autellix: An efficient serving engine for llm agents as general programs," *CoRR*, vol. abs/2502.13965, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2502.13965

[43] Z. Pan, A. Patel, Z. Hu, Y. Shen, Y. Guan, W.-L. Li, L. Qin, Y. Wang, and Y. Ding, "Kvflow: Efficient prefix caching for accelerating llm-based multi-agent workflows," *CoRR*, vol. abs/2507.07400, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2507.07400

[44] Z. Bian, F. Wu, T. Ma, and Y. Zhuo, "Tokencake: A kv-cache-centric serving framework for llm-based multi-agent applications," *CoRR*, vol. abs/2510.18586, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.18586

[45] J. Yang, B. Hou, W. Wei, Y. Bao, and S. Chang, "Kvlink: Accelerating large language models via efficient kv cache reuse," *CoRR*, vol. abs/2502.16002, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2502.16002

[46] H. Kang, Z. Li, X. Yang, W. Xu, Y. Chen, J. Wang, B. Chen, T. Krishna, C. Xu, and S. Arora, "Thunderagent: A simple, fast and program-aware agentic inference system," *CoRR*, vol. abs/2602.13692, 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2602.13692

[47] Y. Jiang, F. Fu, X. Yao, G. He, X. Miao, A. Klimovic, B. Cui, B. Yuan, and E. Yoneki, "Demystifying cost-efficiency in llm serving over heterogeneous gpus," *CoRR*, vol. abs/2502.00722, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2502.00722

[48] X. Cui, C.-J. M. Liang, J. Xing, and H. Qiu, "From models to operators: Rethinking autoscaling granularity for large generative models," *CoRR*, vol. abs/2511.02248, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2511.02248

[49] L. Zhang, M. Chen, R. Cao, J. Chen, F. Zhou, Y. Xu, J. Yang, Z. Ma, L. Chen, C. Luo *et al.*, "Megaflow: Large-scale distributed orchestration system for the agentic era," *CoRR*, vol. abs/2601.07526, 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2601.07526

[50] H. Oh, K. Kim, J. Kim, S. Kim, J. Lee, D.-s. Chang, and J. Seo, "Exegpt: Constraint-aware resource scheduling for llm inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 369–384. [Online]. Available: https://doi.org/10.1145/3620665.3640383

[51] B. Zhang, Y. Wang, and Z. Li, "BOER: enhancing resource utilization for deep learning inference with hybrid spatial GPU sharing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2025, St. Louis, MO, USA, November 16-21, 2025*. ACM, 2025, pp. 519–532. [Online]. Available: https://doi.org/10.1145/3712285.3759857